

Table of contents

1	Memory & Data	2
1.1	The State	2
1.2	Arrays: The Shelf	2
1.3	Structure of Arrays (SoA)	2

Chapter 1

Memory & Data

Simulating reality requires storing vast amounts of information. Every pixel must know its colour, light velocity, and whether it represents a wall or empty air.

1.1 The State

We call the sum of this information the **State**. In our engine, state resides in RAM (Random Access Memory).

1.2 Arrays: The Shelf

The most efficient way to store a list in a computer is an **Array**. Picture an array as a long shelf where every item has a specific number (an index).

1.3 Structure of Arrays (SoA)

Programmers might typically create an Object for every pixel:

```
// Array of Structures (AoS) - SLOW  
let pixel = { red: 1.0, green: 0.5, velocityX: 0.1 };
```

However, managing 200,000 objects is slow. Every time the computer seeks a new object, it may look in a completely different memory location. This is a **Cache Miss**, a primary cause of software sluggishness.

Instead, we use a single massive array of numbers and use **Offsets** to locate data. This **Structure of Arrays (SoA)** layout keeps related data tightly packed, which the CPU prefers.

i Analogy: The Library Shelf

Imagine a library. * **AoS**: Books are organised by Author. To find all red books, you must check every aisle. * **SoA**: Books are organised by Colour. All red books sit on one shelf, allowing you to grab them instantly.

In `src/engine.js`, we define these “shelves” using the `FIELD` constant.

```
/**
 * Memory layout offsets.
 * Access a pixel's property by: index = (y * width + x) * STRIDE + OFFSET.
 */
const FIELD = {
  R: 0,
  G: 1,
  B: 2, // Spectral Energy (Colour)
  VEL_X: 4,
  VEL_Y: 5, // Momentum Vector (Flow direction)
  DEPTH: 7, // Distance from camera (Topology)
};

const STRIDE = 14; // Total slots per pixel
```

We allocate one giant block of memory for the entire universe:

```
State.lattice = new Float32Array(TOTAL_PIXELS * STRIDE);
```

`Float32Array` is a `TypedArray` that stores only decimal numbers. It is extremely fast.